

Final Project Report

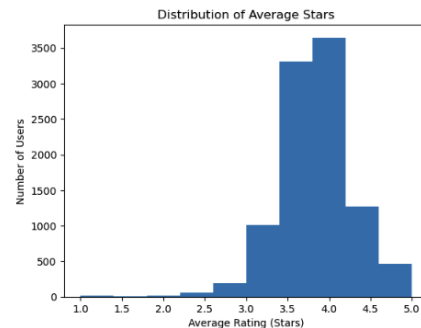
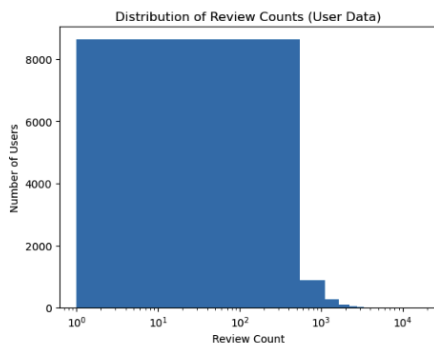
Elena Márquez, Kira Damo, Massiel Payero Garcia

Link to presentation: <https://youtu.be/u6aSM7zRRHo>

Description of the Data

Yelp is an online platform where users can review businesses and rate them on a 5-star scale. Yelp offers a public dataset that consists of nearly 7 million reviews, over 150,000 businesses, and almost 2 million users. The TAR file consists of 5 JSON files with the following data: businesses, reviews, users, check-ins, and tips. Photos are also included in the public dataset. This project implements recommender system models to recommend restaurants to users.

The primary datasets that will be used are businesses, reviews, and users. This subset utilizes a data schema with business ID as a foreign key. The business dataset includes addresses, average stars, review counts, hours, if the business is still open, attributes (whether a business does takeout, parking, music, etc.), and categories (type of business, cuisine, services). Additionally, the reviews data includes the text of reviews of businesses, engagement statistics for each review, and the number of stars given. The last dataset, users, includes information about the review history of users such as review count, age of profile, engagement statistics (compliments, fans, elite status), and average stars they normally rate. Since the Yelp data is extremely large, the data is loaded in batches of 4,000 – 10,000 (depending on model) for practical home computer computation.



Histograms of the distribution of review counts and average stars foreshadow any challenges that will occur in the project as well as provide context for evaluation results. The distribution of the review counts of the users data is highly right skewed indicating that most users leave less than 10 reviews. Users generally leaving less than 10 reviews exhibits sparsity in the data and potential “cold start” issues that commonly occur in recommender systems due to insufficient user ratings. On the other hand, the average amount of stars that each user rated is left skewed. The majority of the stars are between 3 to 4.5 stars. A higher positive bias reduces variance and makes personalization more challenging to

implement. This may cause difficulties with restaurants outranking each other because there is little distinction between good and great restaurants for the user.

To build a restaurant recommendation system, the data frames must only contain restaurants that are still open for business. The variables are pruned by filtering the businesses whose categories contain "restaurant" and have an "is_open" variable return as true. The reviews and users dataset are then narrowed down to the pertaining businesses.

Approach

Content-Based:

The dataset was filtered to include only users with at least ten reviews, keeping only the most recent review per restaurant. This yielded 3,755 reviews of 1,738 businesses from 119 users. Review texts were tokenized and vectorized with $\text{max_df}=0.8$ and $\text{min_df}=2$, resulting in a vocabulary size of 7,115. Adjusting max_df had no effect, but increasing min_df significantly reduced vocabulary size. A cosine similarity matrix ($3,755 \times 3,755$) was created from the vectorized reviews.

Two of the content-based recommenders are non-personalized. The first takes as the input a review index and recommends the k most similar businesses based on the similarity matrix. The second takes as input a business id. It considers similarities from multiple reviews and recommends the k most similar businesses based on review content. Since the non-personalized recommenders do not utilize any user interaction data, typical evaluation metrics cannot be used to determine recommendation relevance.

The personalized user-based recommender creates user profiles by averaging the text vectors of reviews written by each user. Reviews above each user's mean rating are split evenly: half for profile building and half for testing. The system recommends the top k businesses most similar to the user profile, evaluated using precision and recall against the test set.

The last user-based content-based recommender system converts all attributes and categories of each restaurant from the filtered data into dummy variables. User profiles are aggregated similarly and are multiplied by the stars of the review of each applicable restaurant to add weight. Cosine similarity is then used to recommend restaurants similar to the restaurants the user reviewed. To remedy the cold start issue since the data is sparse, the restaurants with the most reviews are recommended. This model is then evaluated using the leave-one-out method and measures the recall and precision as the number of recommendations increases.

Sentiment Analysis: RoBerta

The layer stack in the first version of this sentiment analysis was: 1. **Input** raw review text, 2. **RoBERTa Base**: Processes text to output feature vector embeddings, 3. **Dropout layer**: randomly zero out some neurons during training so neurons don't become dependent on each other, 4. **Linear layer**: Transforms feature vector into single output node. Performing raw linear regression, 5. **Sigmoid activation** function to ensure rating is always a valid decimal.

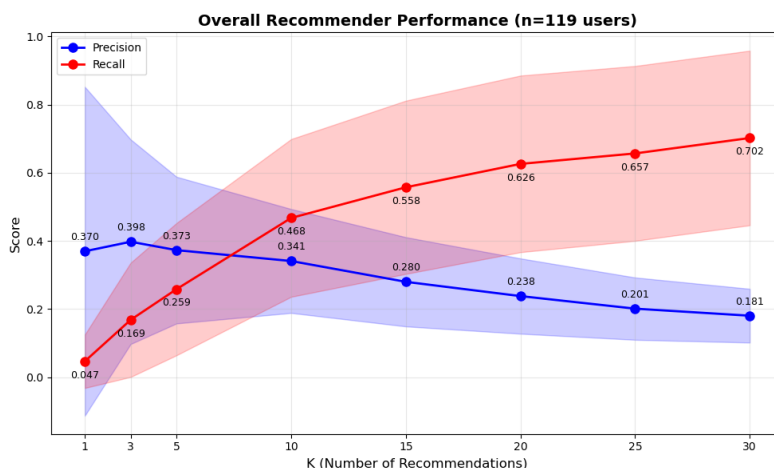
The output is then "stretched" from the sigmoid function to fall within our 5 star range with the following **formula: rating = 1 + (sigmoid_output * 4)**. This neural network is a regression architecture versus classification. Even though the type of system we want to build considers the nuances of ratings with decimal points, the Yelp dataset lends itself better to a classification architecture as a user has to choose between 5 categories, a number between 1-5. Therefore, the *new neural network* has a classification head and has the following layer stack: 1. **Input** raw review text and map stars to labels, 2. **Tokenization of head + tail** (if the review is longer than 512 tokens, we keep the first 256 + last 254 tokens and produce input_ids and attention_mask which tell RoBERTa which tokens are real and which are padding, 3. **RoBERTa Base**: Processes text to output feature vector embeddings, 4. Pooling layer: summarize sequence of vectors into a single vector. We chose mean pooling because we are doing head + tail tokenization. This way both head and tail contribute to the final summary, 5. Dropout: Applied directly to RoBERTa embedding. Prevents over-relying on a few strong embedding features, 6. Linear, 7. Relu: Add non-linearity so two linear layers don't collapse to a single layer transform, 8. Dropout, 9. Linear. The final layer outputs 5 logits. Similar to before, we froze the RoBERTa weights to account for computational overhead. We also only used 2k records to train the data and 55k records to test. At the time of this experiment, we are still running it locally.

To establish a baseline, we trained the classification model using the best hyperparameters found via randomized search cross-validation on the regression model. The search used an 80/20 stratified split, 5 trials, 3 folds, 3 epochs, and batch size 8, yielding: LR = 7.35e-4, Weight Decay = 0.0222, Dropout = 0.163. However, optimal regression hyperparameters don't automatically transfer to classification, as regression uses MSE on continuous values while classification uses cross-entropy on logits.

Results

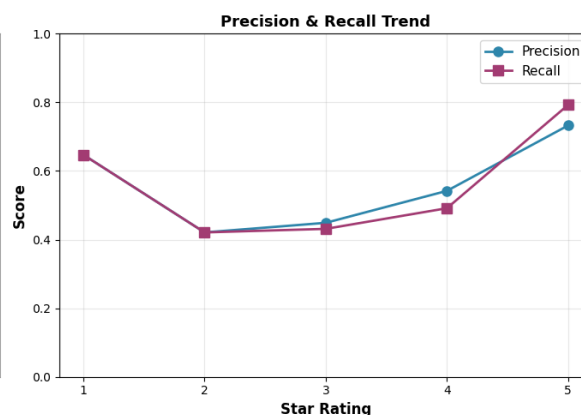
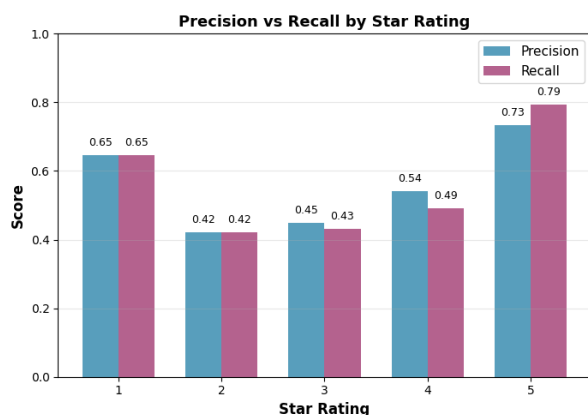
For the non-personalized content-based recommenders, several review indices and business ids were queried, and their outputs were examined. Based on the content of the outputs, it appeared that the recommendations were very relevant to the queries in terms of their content. For the personalized content-based recommender, the mean precision and

recall values were calculated across recommendations for all users at all values of k . Results are not surprising. As k increases, recall increases and precision decreases. For about 37% of users, the top recommended restaurant was in the test set. The best balance for precision and recall appears to be around $k=7$ or $k=8$.



The last K-Nearest Neighbors content-based model follows a similar trend of high recall and low precision as K increases. This model was built on a randomized test and training review dataset. The drawback of this method is that since the business' dataset includes chain restaurants, redundant businesses in different cities can be recommended. The model was evaluated using the Leave-One-Out method since the data is highly sparse.

The RoBERTa classification model achieved 61% accuracy on 400 test reviews, with moderate average per-class performance (56%). It performs best on extreme ratings (1 and 5 stars), partly due to class imbalance (more 4/5-star reviews in Yelp). Most misclassifications were off by only one star, better than random guessing. These results confirm that regression-optimized hyperparameters don't translate optimally to classification.



Confusion Matrix:						Classification Report (classes 1-5 stars):				
	Pred_1	Pred_2	Pred_3	Pred_4	Pred_5		precision	recall	f1-score	support
Actual_1	22	9	2	0	1	1	0.65	0.65	0.65	34
Actual_2	8	16	11	2	1	2	0.42	0.42	0.42	38
Actual_3	3	11	22	14	1	3	0.45	0.43	0.44	51
Actual_4	1	2	14	58	43	4	0.54	0.49	0.52	118
Actual_5	0	0	0	33	126	5	0.73	0.79	0.76	159
						accuracy			0.61	400
						macro avg	0.56	0.56	0.56	400
						weighted avg	0.60	0.61	0.61	400

To see if 1) spending the time to train a neural network was worth it and 2) if the minimal amount of training we did made a big enough difference, we also did a zero-shot inference call to RoBERTa-large-mnli from hugging face and ran it on the same test set. The zero-shot model struggled tremendously when it came to predicting middle classes. It is good to see that even the minimal training we did to help the classification head learn restaurant review specific language patterns helped. This gives us even more motivation to run this experiment in the cloud with more data. Below are the results.

Classification Report (Zero-Shot):								
	precision	recall	f1-score	support				
1	0.45	0.74	0.56	34				
2	0.10	0.55	0.18	38				
3	0.10	0.08	0.09	51				
4	0.67	0.05	0.09	118				
5	0.78	0.47	0.58	159				
accuracy			0.33	400	Metric	Trained Classification	Zero-Shot RoBERTa	Improvement
macro avg	0.42	0.38	0.30	400	Accuracy	0.61	0.33	+28 pts
weighted avg	0.57	0.33	0.34	400	Weighted F1	0.61	0.34	+27 pts
					Macro F1	0.56	0.30	+26 pts

Conclusion

The algorithms implemented in this project demonstrate the various elements of restaurant recommendation that go beyond sponsorships and location proximity. This project only scratches the surface of the full Yelp recommendation algorithm. Incorporating all of the available data such as location and photos would elevate our project. If time allowed, the project would evolve into recommending restaurants that cater to dietary needs such as veganism, gluten-free, dairy-free, etc., since the data has a plethora of attributes and categories.

During this project, we learned just how computationally expensive it is to train and test models. The size of the data was huge, and the data itself was highly sparse. This made computation and evaluation difficult. Common methods used for performance metrics such as cross validation were not viable for these models or this data. In the future, we could explore setting up a cloud environment with GPUs for this type of testing. Beyond the constraints of time and hardware, the project has great potential for a more refined recommendation of restaurants.